

课程笔记

KAIST CS492D Lecture 12: Inference-Time Guided Generation 2

基于 Minhyuk Sung 授课内容整理

2025 年秋季

CS492C: Diffusion and Flow Models

Inference-Time Guided Generation 2

LECTURE 13
MINHYUK SUNG

Fall 2025
KAIST

CS492(C): Diffusion and Flow Models (Fall 2025)

视频作者/频道: Minhyuk Sung (KAIST)

发布日期: 2025

视频时长: 68 分钟

视频链接: <https://www.youtube.com/watch?v=5D95I30oedA>

目录

1 课程回顾与问题背景	3
1.1 DPS 方法的核心近似	3
1.2 DPS 的实现简洁性	3
1.3 本章小结	4
2 SDE/ODE 视角下的引导生成	4
2.1 SDE 表述中的引导	4
2.2 ODE 表述的可行性	5
2.3 时间步数与引导强度的权衡	5
2.4 本章小结	6
3 Test-Time Scaling: 从 LLM 到扩散模型	6
3.1 预训练时代的终结与推理时计算	6
3.2 从 LLM 到扩散模型的迁移	6
3.3 RL 视角下的推理时引导	7
3.4 本章小结	7
4 奖励函数引导: RL 风格的推导	7
4.1 问题设定: 带奖励的采样	7
4.2 KL 约束下的奖励最大化	8
4.3 最优分布的闭式解	8
4.4 中间时间步的分布与最优价值函数	8
4.5 价值函数的 Tweedie 近似	9
4.6 得到 DPS 等价公式	9
4.7 本章小结	10
5 超越逆问题: 多样化应用	10
5.1 图像领域的拓展应用	10
5.2 全景图像生成	10
5.3 360 度全景图像	11
5.4 3D 纹理生成	11
5.5 歧义图像 (Visual Illusions) 生成	12
5.6 无限缩放 (Infinite Zooming)	12
5.7 运动生成的长序列拼接	13
5.8 本章小结	13
6 方法的优势与局限	13
6.1 推理时引导方法的全面比较	13
6.2 优势	13
6.3 局限	14
6.4 本章小结	14

7 实践落地：如何把推理时引导做成可复现实验	14
7.1 实验设计的四个关键旋钮	14
7.2 推荐的实验日志模板	15
7.3 从单奖励到多奖励组合	15
7.4 研究前沿：从 hand-crafted guidance 到 learned verifier	16
7.5 评测协议：不要只看单一视觉指标	16
7.6 为什么这条路线对中小团队尤其重要	16
7.7 本章小结	17
8 总结与延伸	17
8.1 核心要点回顾	17
8.2 与 LLM Test-Time Scaling 的呼应	17
8.3 总结表：方法、价值与风险	17
8.4 给研究者的三个行动点	18
8.5 进一步延伸的问题	18
8.6 与训练时控制方法的分工	18
8.7 拓展阅读	18

1 课程回顾与问题背景

本节课是 KAIST CS492D “扩散模型专题” 系列第 12 讲，承接上一讲关于推理时引导生成 (Inference-Time Guided Generation) 的内容，进一步深入讨论更高级的引导技术。上一讲主要介绍了三类推理时引导方法：

1. **注意力与中间层操纵**：通过直接修改神经网络的中间层或注意力输出来实现引导。优点是实现简单，缺点是缺乏理论解释，且仅适用于特定场景。
2. **基于逆问题的引导 (DPS)**：将条件生成建模为逆问题，通过贝叶斯法则分解条件分数为边际分数与条件似然梯度之和。
3. **基于奖励函数的引导**：使用可微的奖励函数定义用户偏好，在生成过程中注入梯度引导。

逆问题回顾

给定观测量 y 和映射函数 $\mathcal{A}$ ，逆问题的目标是找到满足 $y = \mathcal{A}(x)$ 的 x 。在扩散模型框架下，我们需要计算条件分数 $\nabla_{x_t} \log p(x_t|y)$ ，利用贝叶斯法则将其分解为：

$$\nabla_{x_t} \log p(x_t|y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y|x_t)$$

其中第一项是预训练模型已经学到的边际分数，第二项是需要额外计算的条件似然梯度。

1.1 DPS 方法的核心近似

DPS (Diffusion Posterior Sampling) 方法的关键思路是：当逆问题中的映射函数 $\mathcal{A}$ 为线性时，条件分数项可以通过参数 R ($\mathcal{A}$ 的伪逆) 来计算。更进一步，可以通过更粗糙的近似——将条件分布近似为高斯分布——来得到更简洁的形式：

$$\nabla_{x_t} \log p(y|x_t) \approx -\nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0(x_t))\|^2$$

其中 $\hat{x}_0(x_t)$ 是由当前噪声状态 x_t 通过 Tweedie 公式估计的干净数据。

Tweedie 公式

Tweedie 公式给出了在给定噪声观测 x_t 时，对干净数据 x_0 的最优后验估计：

$$\hat{x}_0(x_t) = \frac{1}{\sqrt{\bar{\alpha}_t}} (x_t + (1 - \bar{\alpha}_t) \nabla_{x_t} \log p(x_t))$$

在实践中，这等价于用预训练的噪声预测网络 $\epsilon_\theta(x_t, t)$ 来估算 x_0 。

1.2 DPS 的实现简洁性

DPS 方法在实现上极其简单——只需在生成流水线中添加一行代码。在每个去噪步骤中，先通过标准的反向过程计算 x_{t-1} ，然后根据 L2 损失的梯度对 x_{t-1} 进行一步更新：

$$x_{t-1} \leftarrow x_{t-1} - \eta \cdot \nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0(x_t))\|^2$$

其中 η 是引导步长 (guidance step size)，控制每步引导的强度。整个 DPS 算法的流程可以总结为：

1. 从 $x_T \sim \mathcal{N}(0, I)$ 开始
2. 对每个时间步 $t = T, T-1, \dots, 1$:
 - (a) 用预训练模型计算 $\hat{x}_0(x_t)$ (Tweedie 估计)
 - (b) 计算引导梯度 $g = \nabla_{x_t} \|y - \mathcal{A}(\hat{x}_0(x_t))\|^2$
 - (c) 执行标准去噪步得到 $\tilde{x}_{t-1}$
 - (d) 应用引导更新 $x_{t-1} = \tilde{x}_{t-1} - \eta_t \cdot g$
3. 输出 x_0

讲者强烈建议学生用自己的扩散模型代码（如课程作业中的实现）来实验 DPS 方法，因为它实际上只需要在生成流水线中添加一行梯度更新代码。

近似的代价

虽然将 $p(y|x_t)$ 近似为高斯分布使得计算极为简便，但这并非真实分布。尤其是在高噪声时间步， $\hat{x}_0$ 的估计质量较差，条件似然的近似误差会很大。因此实际效果往往对引导权重和时间步数非常敏感。

线性逆问题 vs. 非线性逆问题

当映射函数 $\mathcal{A}$ 为线性时（如超分辨率、去模糊、修复等），可以推导出更精确的条件分数表达式，涉及 $\mathcal{A}$ 的伪逆运算。但在实践中，即使是线性情况下，DPS 的简单高斯近似也能取得不错的效果，因此通常直接使用通用的 L2 梯度引导形式。对于非线性映射 $\mathcal{A}$ （如相位恢复、非线性传感器模型），高斯近似的误差可能更大，但仍是目前最实用的方法。

1.3 本章小结

DPS 框架通过贝叶斯法则将条件分数分解为边际分数和条件似然梯度，并利用高斯近似将后者简化为 L2 损失的梯度。这一方法不需要额外训练，可以应用于任意可微的条件函数。

2 SDE/ODE 视角下的引导生成

2.1 SDE 表述中的引导

扩散模型的反向过程可以等价地表述为连续时间的随机微分方程 (SDE)。在 SDE 表述下，反向过程的时间域离散化形式为：

$$x_{t-\Delta t} = x_t + f(x_t, t)\Delta t + g(t)\Delta W$$

其中 f 包含分数函数， g 是噪声系数。将 DPS 引导引入 SDE 表述同样是直接的：在每一步计算 $x_{t-\Delta t}$ 后，根据条件梯度进行更新。

SDE 与 DDPM 的等价性

DDPM 的离散反向步骤可以视为对连续 SDE 的时间域离散化。两种表述完全等价——SDE 表述提供了更自然的连续时间框架，而 DDPM 是其在离散时间步上的特例。DPS 引导可以在两种表述中一致地应用。

2.2 ODE 表述的可行性

除了 SDE 表述外，扩散模型的反向过程也可以用概率流 ODE 来描述。概率流 ODE 的形式为：

$$\frac{dx}{dt} = f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x)$$

其特点是没有随机噪声项，给定初始条件 x_T 的轨迹是确定性的。课堂讨论表明：DPS 的引导思路完全可以应用到 ODE 表述中，只需在每步求解后添加引导梯度更新。

三种表述的对比

特性	DDPM	SDE	ODE
时间域	离散	连续	连续
随机性	有	有	无
采样质量	高	高	高
采样速度	慢（需多步）	慢	可加速
DPS 引导	支持	支持	支持
理论基础	变分推断	随机分析	概率流

三种表述在引导方面面临相同的实际问题：Tweedie 近似的精度和引导强度的权衡。

2.3 时间步数与引导强度的权衡

更多时间步 vs. 更大引导权重

推理时引导存在一个核心权衡：

- **增加时间步数**：更多步意味着更多次注入引导梯度，引导效果更好，但生成速度变慢。
- **增大引导权重**：减少时间步但增大每步的引导强度，会导致中间样本偏离训练分布，使得噪声预测网络的预测变得不准确，最终输出变得不真实。

这是推理时引导方法的核心难题：输出要么不满足给定条件，要么满足条件但变得不真实。如何在真实性与条件满足之间取得平衡，是调参的关键。

如果希望同时获得加速（少步数）和良好引导效果，可能需要在训练阶段就引入条件信息。但这就失去了推理时引导“无需额外训练”的优势。

讲者指出，这一权衡在实践中表现为：当引导权重过大时，中间样本 x_t 会被推到训练数据分布之外的区域。在这些分布外区域，噪声预测网络 ϵ_θ 从未见过类似的输入，因此其预测会变得不准确。这种不准确的预测会在后续步骤中累积，导致最终输出虽然可能满足给定条件，但在视觉上变得不真实、产生伪影。

实用调参建议

根据课堂讨论，以下经验法则有助于在实践中取得较好的结果：

- 使用较多的去噪步数（如 1000 步），让每步引导的扰动较小
- 引导权重可以随时间步变化：在早期（高噪声）步使用较小的权重，在后期（低噪声）步逐渐增大
- 同时尝试 SDE 和 ODE 两种采样器，比较结果
- 对于同一个条件，多次采样取最优（即 best-of-N 策略）

2.4 本章小结

DPS 引导可以在 DDPM、SDE 和 ODE 三种表述中一致应用。实践中需要仔细调整时间步数和引导权重，以在条件满足度和输出真实性之间取得平衡。

3 Test-Time Scaling: 从 LLM 到扩散模型

3.1 预训练时代的终结与推理时计算

Ilya Sutskever（OpenAI 联合创始人）曾提出一个影响深远的观点：预训练时代正在走向终结。对于大语言模型（LLM）而言，互联网上可用的训练数据已经接近用尽。即使继续扩大模型规模或增加训练计算量，性能提升也将越来越有限。

Test-Time Scaling 的兴起

Test-time scaling 的核心思想是：与其在训练时投入更多计算资源，不如在推理时花费更多计算来提升输出质量。这一思想在 LLM 领域已经取得了显著成功：

- OpenAI 的 O1/O3 模型通过“思考模式”在推理时花费更多时间来改善答案
- DeepSeek R1 通过 RL 训练模型产生中间思考 token，实现了“aha moment”——模型能自我发现并纠正推理错误
- 在 ARC-AGI 基准测试中，O3 的分数从此前模型的 10%–20% 跃升至 88%

3.2 从 LLM 到扩散模型的迁移

课程的核心问题是：能否将 test-time scaling 的思想从 LLM 迁移到扩散模型和流模型？讲者认为这是一个极具前景的方向，且相比训练时计算，推理时技术对计算资源的要求更低，这使得小规模研究团队也有机会开发出性能优异的模型。

Proposer-Verifier 框架

Test-time scaling 的基本架构由两个组件构成：

- **Proposer (提议者)**：预训练的生成模型（如扩散模型），负责生成候选输出。
- **Verifier (验证者)**：另一个模型或函数，评估 proposer 的输出质量。可以是神经网络、代码验证器或任何评分函数。

基于 verifier 的反馈，可以（1）引导 proposer 的生成过程，或（2）让 proposer 生成多个候选，从中选择最优。这与 GAN 的“生成器-判别器”框架异曲同工。

3.3 RL 视角下的推理时引导

Dan（OpenAI 工程师）的演讲生动阐述了 test-time scaling 的愿景：就像 1907 年的爱因斯坦需要 8 年才能发展出广义相对论，而 O3 模型只需要一分钟的“思考”就能解答广义相对论期末考试题——这依赖于推理时花费更多计算。

Yann LeCun 曾将预训练比作“大蛋糕”，强化学习只是“蛋糕上的樱桃”。但 OpenAI 的路线图表明，未来 RL 计算将在总计算中占据越来越大的比重，最终可能远超预训练计算。

研究趋势的启示

讲者观察到，当前 AI 研究社区已不再仅仅关注开发单一新技术或发表论文。越来越多的研究团队（包括创业公司）正在利用 test-time scaling 技术，用相对较少的计算资源（相比从头训练大模型）来超越现有模型的性能。这为资源有限的学术实验室和小团队提供了重要机会——不需要拥有数千张 GPU，也可以通过巧妙的推理时技术取得竞争力强的结果。

3.4 本章小结

Test-time scaling 已经在 LLM 领域展现了巨大潜力。将这一思想迁移到扩散模型，通过 proposer-verifier 框架在推理时提升生成质量，是当前研究的重要方向。

4 奖励函数引导：RL 风格的推导

4.1 问题设定：带奖励的采样

假设我们有一个预训练的扩散模型，其学到的数据分布为 $p_0(x_0)$ 。同时我们有一个连续可微的奖励函数 $r(x_0)$ ，当输出 x_0 更符合用户偏好时， $r(x_0)$ 值更高。

逆问题是奖励函数的特例

所有逆问题都可以视为奖励函数引导的特例。只需定义：

$$r(x_0) = -\|y - \mathcal{A}(x_0)\|^2$$

即 L2 损失的负值作为奖励函数。当输出 x_0 通过映射 $\mathcal{A}$ 后越接近观测 y ，奖励越高。

奖励函数引导的适用范围远超逆问题。例如：

- **运动生成**：定义奖励函数使得生成的运动轨迹避免与场景中的物体碰撞
- **分子生成**：定义奖励函数使得生成的分子结构具有特定的对接性质
- **任意可微判别模型**：只要有一个可微的评分模型，就可以将其输出作为奖励函数

4.2 KL 约束下的奖励最大化

我们希望定义一个新的数据分布 $p_0^*(x_0)$ ，使得从中采样的 x_0 不仅具有高奖励，还不会偏离原始分布太远。形式化为如下优化问题：

$$p_0^* = \arg \max_p \mathbb{E}_{x_0 \sim p}[r(x_0)] - \frac{1}{\beta} D_{\text{KL}}(p \| p_0)$$

其中 $\beta > 0$ 控制奖励最大化与分布偏离之间的权衡：

- β 越大，越侧重于奖励最大化，但可能产生不真实的样本
- β 越小，越接近原始分布，但奖励可能不够高

与 RLHF 的联系

这一公式形式与 LLM 领域的 RLHF (Reinforcement Learning from Human Feedback) 完全一致。在 RLHF 中， p_0 是 SFT 模型的分布， r 是奖励模型的输出， β 是 KL 惩罚系数。扩散模型的推理时引导和 LLM 的 RLHF 本质上在解决同一个数学问题。

4.3 最优分布的闭式解

上述优化问题有闭式解。通过拉格朗日乘子法和变分计算，可以证明最优分布为：

$$p_0^*(x_0) = \frac{1}{Z_0} p_0(x_0) \exp(\beta \cdot r(x_0))$$

其中 $Z_0 = \int p_0(x_0) \exp(\beta \cdot r(x_0)) dx_0$ 是归一化常数（配分函数）。可以看到，新分布在原始分布的基础上，按照 $\exp(\beta \cdot r(x_0))$ 进行重加权——高奖励区域的概率密度被放大，低奖励区域被压缩。

能量模型的视角

$p_0^*(x_0)$ 的形式恰好是一个能量模型（Energy-Based Model），其能量函数为：

$$E(x_0) = -\log p_0(x_0) - \beta \cdot r(x_0)$$

原始数据分布提供了“先验能量”，奖励函数提供了“条件能量”。 β 控制了二者的相对权重，类似于统计物理中的逆温度参数。当 $\beta \rightarrow 0$ 时， $p_0^* \rightarrow p_0$ （回到原始分布）；当 $\beta \rightarrow \infty$ 时， p_0^* 集中在奖励最大的点附近。

4.4 中间时间步的分布与最优价值函数

核心挑战：中间时间步

扩散模型的生成过程涉及从 x_T 到 x_0 的多步去噪。为了从 p_0^* 采样，我们不仅需要知道 p_0^* ，还需要知道每个中间时间步的分布 $p_t^*(x_t)$ 。这要求我们定义并计算最优价值函数 $V^*(x_t, t)$ 。

类比于最终分布中奖励函数 $r(x_0)$ 的角色，中间时间步的分布可以表示为：

$$p_t^*(x_t) = \frac{1}{Z_t} p_t(x_t) \exp(\beta \cdot V^*(x_t, t))$$

其中 $V^*(x_t, t)$ 是最优价值函数 (optimal value function)，表示从 x_t 出发、遵循最优策略所能获得的期望未来奖励。

价值函数没有解析解

与最终分布不同，中间时间步的最优价值函数 $V^*(x_t, t)$ 没有解析形式。这是因为它涉及对所有可能未来轨迹的期望，无法直接计算。因此必须引入近似。

4.5 价值函数的 Tweedie 近似

借鉴 DPS 中的 Tweedie 近似思路，可以将最优价值函数近似为：

$$V^*(x_t, t) \approx r(\hat{x}_0(x_t))$$

即用当前时刻对 x_0 的 Tweedie 估计代入奖励函数。这一近似的含义是：中间时间步的“未来期望奖励”可以用“当前最优估计的即时奖励”来代替。

虽然这是一个相当粗糙的近似，但它具有重要的实用价值：

- 不需要训练额外的价值网络
- 只需利用已有的扩散模型（提供 Tweedie 估计）和奖励函数
- 计算开销仅为一次前向传播（Tweedie 估计）+ 一次奖励函数评估 + 一次反向传播（计算梯度）

讲者也提到，理论上可以训练一个专门的神经网络来学习最优价值函数 $V^*(x_t, t)$ ，但这种方法在实践中往往不够稳定。因此 Tweedie 近似虽然简单，却是目前最实用的方案。

价值函数近似的层级

从精确到粗糙，价值函数有以下几种近似方式：

1. **精确计算**：需要对所有未来轨迹求期望，计算不可行
2. **训练价值网络**：训练 $V_\phi(x_t, t)$ 近似 V^* ，需要大量训练数据和时间，且可能不稳定
3. **蒙特卡洛估计**：从 x_t 出发采样多条轨迹，取平均奖励。计算量大但更准确
4. **Tweedie 单步估计**： $V^*(x_t, t) \approx r(\hat{x}_0(x_t))$ ，最简单但误差最大

DPS 和大部分推理时引导方法采用第 4 种，因为它不需要任何额外训练。

4.6 得到 DPS 等价公式

在上述近似下，新分布的分数函数变为：

$$\nabla_{x_t} \log p_t^*(x_t) = \underbrace{\nabla_{x_t} \log p_t(x_t)}_{\text{原始分数}} + \underbrace{\beta \cdot \nabla_{x_t} r(\hat{x}_0(x_t))}_{\text{期望未来奖励的梯度}}$$

DPS 的 RL 解释

将奖励函数取为 $r(x_0) = -\|y - \mathcal{A}(x_0)\|^2$ ，上式立即退化为 DPS 的公式。因此，DPS 可以从两个角度理解：

- **贝叶斯视角**：条件分数 = 边际分数 + 条件似然梯度
- **RL 视角**：最优策略分数 = 原始策略分数 + 期望未来奖励梯度

两者在数学上完全等价，但 RL 视角提供了更一般的框架，奖励函数不限于逆问题。

4.7 本章小结

通过 KL 约束的奖励最大化，我们可以定义一个新的目标分布。利用 Tweedie 近似估计最优价值函数后，可以推导出与 DPS 完全等价的引导公式。RL 视角为推理时引导提供了更一般的理论框架。

5 超越逆问题：多样化应用

5.1 图像领域的拓展应用

推理时引导的威力远不止于解决逆问题。只要能定义合适的可微奖励函数，就可以引导预训练模型生成满足各种条件的输出。以下是几个典型应用场景。

从逆问题到通用引导

逆问题只是奖励函数引导的一个特例。通用引导框架的适用条件是：

1. 拥有一个预训练的扩散/流模型（proposer）
2. 能够定义一个可微的奖励/损失函数（verifier）
3. 奖励函数能编码用户希望输出满足的条件

满足这三个条件的任何问题都可以使用推理时引导。

5.2 全景图像生成

预训练的图像扩散模型通常只能生成固定分辨率（如 512×512 ）的图像。如何利用预训练模型生成更大尺寸的全景图像？

核心思路：将大画布划分为多个重叠的窗口（每个窗口大小等于模型训练分辨率），在所有窗口上并行执行去噪过程，同时通过引导确保重叠区域一致。

具体而言：

1. 在大画布上放置多个重叠的窗口
2. 对每个窗口独立执行扩散去噪

3. 定义奖励函数：相邻窗口在重叠区域应产生相同的像素值
4. 奖励函数可以简单定义为重叠区域的 L2 距离（或更高级的风格损失）
5. 在每步去噪后，根据重叠一致性的梯度更新所有窗口

Canonical Space 与 Instance Space

这类同步化应用引入了两个概念：

- **Canonical Space (规范空间)**：最终生成数据所在的空间（如全景图像的完整画布、3D 物体的纹理空间）
- **Instance Space (实例空间)**：每个扩散过程独立操作的空间（如单个窗口、单个视角的 2D 图像）

生成过程在多个 instance space 中并行进行，通过 canonical space 中的一致性约束实现同步。

实际实现中，最简单的同步化方式（与 DPS 完全一致）是：

1. 在 canonical space 中采样所有 x_T
2. 将 canonical space 中的噪声投影到各 instance space
3. 在各 instance space 中独立进行一步去噪
4. 将去噪结果投影回 canonical space
5. 在重叠区域取平均（或根据 L2 损失梯度更新）
6. 将更新后的结果再投影回各 instance space，继续下一步

简单平均 vs. 梯度引导

最朴素的同步化方式是直接在重叠区域取平均。这种方式简单但可能导致模糊和不一致。使用 DPS 风格的梯度引导（定义 L2 一致性损失或风格损失作为奖励函数）通常能取得更好的结果，因为它让模型在每步去噪时就“意识到”一致性约束的存在。

5.3 360 度全景图像

同样的思路可以拓展到 360 度全景图像的生成。将球面展开为多个重叠的平面投影，在每个投影上独立运行扩散过程，通过球面上的重叠一致性约束实现全局协调。

5.4 3D 纹理生成

给定一个 3D 模型的表面几何（mesh），如何利用 2D 图像扩散模型为其生成纹理？这是一个极其实用的问题——3D 模型在游戏、电影、虚拟现实无处不在，而手工制作纹理耗时费力。

1. 从多个视角对 3D 表面进行渲染（投影到 2D），每个视角得到一张部分可见的 2D 图像
2. 在每个视角的 2D 图像上运行扩散去噪（instance space 中的操作）
3. 将 2D 结果反投影回 3D 表面（映射回 canonical space）

4. 通过奖励函数确保不同视角在 3D 表面上的重叠区域一致

这里 canonical space 是 3D 表面上的纹理空间 (UV space), instance space 是各个视角的 2D 图像空间。投影函数 π_i 将 3D 表面上的点映射到第 i 个视角的 2D 坐标, 反投影函数 π_i^{-1} 将 2D 像素映射回 3D 表面。

3D 纹理生成的同步化约束

设从 K 个视角 $\{v_1, \dots, v_K\}$ 观察 3D 物体, 每个视角 v_i 的 2D 渲染图像为 I_i 。对于 3D 表面上任意一个点 p , 如果它在视角 v_i 和 v_j 中都可见, 则需要满足:

$$\pi_i(p) \text{ 处的颜色} = \pi_j(p) \text{ 处的颜色}$$

这一约束可以形式化为奖励函数:

$$r = - \sum_{i < j} \sum_{p \in \text{overlap}(v_i, v_j)} \|I_i(\pi_i(p)) - I_j(\pi_j(p))\|^2$$

5.5 歧义图像 (Visual Illusions) 生成

一个有趣的应用是生成“歧义图像”——一张图像在不同变换下呈现完全不同的含义:

- 正放是一群人的油画, 翻转后变成一位老人的肖像
- 正放是一盆绿植, 翻转后变成爱因斯坦的画像

歧义图像的形式化

生成歧义图像可以视为同步化问题:

- Canonical space: 原始图像空间
- Instance spaces: 原始视角和变换后视角 (如翻转、旋转)
- 约束: 在 canonical space 中只有一张图像, 但它在两个 instance space 中分别符合不同的文本描述

5.6 无限缩放 (Infinite Zooming)

另一个令人印象深刻的应用是利用图像扩散模型生成“无限缩放”视频——从星系不断放大到盘子上的细节再到建筑物内部。这可以通过定义不同缩放级别的 instance space, 并在相邻级别的重叠区域施加一致性约束来实现。

具体地, 每个缩放级别对应一个 instance space, 相邻级别之间的“重叠区域”是高缩放级别中的整张图像与低缩放级别中的中心区域。通过确保这些区域的一致性, 就能生成视觉上连贯的无限缩放效果。

无限缩放的分层结构

无限缩放可以看作分层级的同步化问题：

- 第 0 层：最宏观的视图（如整个星系）
- 第 1 层：第 0 层中心区域的放大版本
- 第 2 层：第 1 层中心区域的进一步放大
- ...

每一层都是一个独立的 instance space，使用同一个预训练图像模型进行去噪。相邻层之间通过缩放对应关系定义一致性约束，确保放大后的图像内容与上一层的中心区域在视觉上匹配。

5.7 运动生成的长序列拼接

对于运动生成（motion generation），预训练模型通常只能生成短时间序列。可以通过类似的同步化技术，将多段短运动拼接为长运动序列，同时确保过渡平滑。

5.8 本章小结

推理时引导的应用远超逆问题。通过定义合适的奖励函数和同步化约束，可以利用预训练的 2D 图像模型生成全景图、3D 纹理、歧义图像等多种超出训练域的内容。核心思想是 canonical space 与 instance space 的分离与同步。

6 方法的优势与局限

6.1 推理时引导方法的全面比较

方法	注意力操纵	DPS (逆问题)	奖励函数引导
理论基础	无（工程直觉）	贝叶斯法则	RL / KL 约束优化
适用条件	特定架构	已知映射 $\mathcal{A}$	可微奖励函数
通用性	低	中	高
实现难度	中	低	低
训练需求	无	无	无（或少量）
典型应用	布局控制	超分/去噪/修复	任意偏好引导

6.2 优势

1. **通用性**：只要能定义可微的奖励函数，就可以应用。适用于图像、视频、运动、分子等多种模式。
2. **无需额外训练**：利用现有预训练模型，无需收集新数据或进行额外训练。
3. **实现简单**：核心代码只需在生成流水线中添加梯度引导步骤。
4. **灵活组合**：可以同时施加多个奖励函数，实现多条件引导。
5. **计算友好**：相比从头训练，推理时引导只需较少的计算资源。

6.3 局限

奖励函数必须可微

推理时引导依赖于梯度反传，因此奖励函数必须是可微的。对于不可微的评价指标（如 FID、IS 等），不能直接用作引导信号。这是当前方法的一个根本限制。

真实性与条件满足的权衡不可避免

增大引导强度会使中间样本偏离训练分布，导致噪声预测网络在分布外区域产生不准确的预测。最终表现为：

- 引导强度小：输出真实但不满足条件
- 引导强度大：满足条件但输出不真实

寻找最佳引导权重通常需要经验性调参。

其他局限包括：

- 推理速度变慢：更多时间步能提供更好的引导效果，但增加了推理时间
- Tweedie 近似在高噪声步误差较大：早期时间步的 $\hat{x}_0$ 估计质量差
- 没有理论保证收敛到全局最优

与训练时方法的对比

推理时引导与训练时方法（如 ControlNet、条件扩散模型、RLHF 微调）形成互补：

- **训练时方法**：效果更稳定、生成速度快，但需要大量数据和计算资源进行训练，且每种条件需要单独训练
- **推理时方法**：无需训练、灵活性高、可即时组合多种条件，但效果不够稳定、生成速度较慢

在实际工程中，最佳策略往往是二者结合：用训练时方法处理常见的、固定的条件类型，用推理时引导处理临时的、多样化的条件需求。

6.4 本章小结

推理时引导方法具有通用性强、无需训练、实现简单等优势，但受限于奖励函数必须可微以及真实性-条件满足度的固有权衡。

7 实践落地：如何把推理时引导做成可复现实验

7.1 实验设计的四个关键旋钮

从课程内容往工程落地推进时，最重要的不是再发明一个公式，而是把几个核心旋钮系统化管理起来。

1. **采样器选择**：DDPM、SDE、ODE 各自对应不同的速度与稳定性权衡

2. **引导强度调度**: 固定权重最简单, 但常常不如随时间步变化的 schedule 稳定
3. **验证器质量**: reward / verifier 是否可信, 直接决定引导方向是否正确
4. **候选筛选策略**: 单次采样、best-of- N 、重排序 (reranking) 会显著改变最终效果

很多实验失败并不是模型不行, 而是 verifier 不够好

一旦 verifier 本身与人类偏好不一致, 或者只会奖励一些容易作弊的局部模式, 扩散模型就会学会“投机取巧”。这与 RLHF 中 reward hacking 的问题完全同构。

7.2 推荐的实验日志模板

为了让不同方法能被公平比较, 课程内容很适合落成统一的实验记录模板:

实验项	需要记录的字段	说明
基础模型	预训练模型、分辨率、训练域	影响 Tweedie 估计质量与可迁移性
采样设置	步数、采样器、随机种子	决定速度与结果波动范围
引导设置	reward 类型、权重、schedule	决定条件满足度与真实性平衡
验证方式	自动指标、人工偏好、失败样例	防止只看平均分, 忽略模式性崩坏
计算开销	单样本耗时、显存、best-of- N 次数	便于和训练时方法做成本比较

表 1: 推理时引导实验的最小记录模板

为什么“失败案例库”很重要

扩散模型的引导方法往往在平均案例上看起来不错, 但在某些条件组合或高分辨率场景下会系统性失真。为失败样例单独建库, 比单纯汇报均值更有助于下一轮研究。

7.3 从单奖励到多奖励组合

实际应用 rarely 只有一个目标。图像既要符合文本, 又要满足几何一致性、风格约束、编辑指令、物理约束等。于是总奖励常写成:

$$r_{\text{total}}(x) = \lambda_1 r_{\text{text}}(x) + \lambda_2 r_{\text{consistency}}(x) + \lambda_3 r_{\text{physics}}(x) + \dots$$

这带来两个新问题:

- 不同奖励尺度不同, 直接相加会让某一项支配优化
- 各奖励目标可能彼此冲突, 例如更强的几何约束可能压缩图像细节与多样性

多奖励组合最常见的错误：权重凭直觉乱调

如果没有做 reward normalization、梯度裁剪或时间步分配，不同约束会相互干扰，结果既不真实也不满足条件。一个更稳妥的做法是让不同奖励在不同时间段接管主导权，例如前期强调全局结构，后期强调细节和一致性。

7.4 研究前沿：从 hand-crafted guidance 到 learned verifier

讲座的隐含趋势是：短期内，Tweedie 近似配合手工奖励函数仍然是性价比最高的路径；但长期来看，更可能的方向是训练更强的 verifier 或 value model：

- **Learned reward model**：让 verifier 更接近人类偏好或下游任务成功率
- **Temporal value model**：不只评估最终 x_0 ，还评估中间状态 x_t 的未来潜力
- **Adaptive compute**：把更多推理步数分配给困难样本，而不是平均对待所有输入
- **Hybrid pipeline**：训练时控制模型负责大方向，推理时引导负责末端精修

7.5 评测协议：不要只看单一视觉指标

课程虽然以方法推导为主，但从研究方法论上看，推理时引导尤其需要多维评测。一个更完整的 protocol 至少应包含：

1. **条件满足度**：逆问题重建误差、约束满足率、任务成功率
2. **真实性**：FID、CLIP score、人工主观评分或任务特定 perceptual metric
3. **稳定性**：不同随机种子下结果波动是否可控
4. **成本**：每次采样的 wall-clock time、显存、best-of- N 带来的额外开销

只汇报“最好的一张图”会严重夸大方法效果

推理时引导很容易通过多次采样挑出漂亮案例，但如果不报告平均表现、失败率和计算成本，就无法公平比较不同方法。test-time scaling 的优势与代价必须同时披露。

7.6 为什么这条路线对中小团队尤其重要

讲者强调的一个现实判断是：在大模型训练门槛持续升高的背景下，test-time 方法给了中小实验室另一条竞争路径。

- 不需要从头训练大型生成模型
- 可直接复用公开权重与开源采样器
- 研究重点转移到 verifier 设计、奖励构造和 compute allocation
- 更适合快速试错，因为每次实验的反馈周期比重新训练短得多

扩散模型社区可能复制 LLM 社区的演化

LLM 已经走出一条很清晰的路径：预训练模型商品化后，真正拉开差距的是后训练、评测和 test-time orchestration。扩散模型很可能也会沿着类似路线演进，未来竞争重点会越来越偏向 verifier、tooling 和 system design。

7.7 本章小结

把推理时引导真正做成研究或产品系统，关键在于统一实验记录、设计稳定的 reward 组合、并把 verifier 质量视为第一等公民。很多看似算法层的问题，本质上是实验设计和评测设计的问题。

8 总结与延伸

8.1 核心要点回顾

本讲从三个角度深入阐述了推理时引导生成的理论与实践：

- SDE/ODE 视角的统一**：DPS 引导不仅适用于 DDPM 的离散表述，也可以在 SDE 和 ODE 的连续时间表述中一致应用。
- RL 视角的重新诠释**：通过 KL 约束的奖励最大化框架，推导出了与 DPS 等价的引导公式。这一 RL 视角揭示了推理时引导与 RLHF 的深层联系。
- 丰富的应用场景**：通过 canonical space/instance space 的同步化技术，预训练模型可以生成全景图、3D 纹理、歧义图像等远超训练域的内容。

8.2 与 LLM Test-Time Scaling 的呼应

本讲将扩散模型的推理时引导置于更宏大的 AI 趋势中——test-time scaling。LLM 领域通过 O1/O3/DeepSeek R1 展示了在推理时花费更多计算的巨大潜力。扩散模型的推理时引导本质上也是一种 test-time scaling 技术，它不增加训练成本，而是通过推理时的额外计算来提升输出质量。

8.3 总结表：方法、价值与风险

主题	核心价值	主要风险	实践建议
DPS / 逆问题引导	无需重训即可处理观测约束	高噪声阶段近似误差大	采用较多步数并做 guidance schedule
RL 视角奖励引导	把问题统一到 verifier + reward 框架	reward hacking、价值近似粗糙	先做 verifier 校验，再谈更强引导
同步化生成	用 2D 预训练模型外推到 panorama / 3D / illusion	多实例间约束冲突，易失真	在 canonical / instance space 间显式记录投影与重叠区域
Test-time scaling	以较低训练成本换更高推理质量	计算开销上升，延迟更高	用 best-of- N 或自适应算力分配控制成本

表 2: Lecture 12 的方法总览

8.4 给研究者的三个行动点

- 1. 先在一个简单、可微、可评测的 reward 上建立稳定基线，再扩展到多约束设置
- 2. 为每种方法建立统一的失败案例库，特别记录高引导权重与低步数下的退化模式
- 3. 把扩散模型的推理时引导和 LLM 的 verifier / reranking / adaptive compute 方法对照研究，而不是分开看待

8.5 进一步延伸的问题

- 是否能训练一个统一 verifier，同时服务逆问题、编辑任务与美学偏好？
- 是否可以让模型动态决定哪些样本值得更多推理步数，从而做到真正的 adaptive test-time scaling？
- 对于 3D、视频和机器人轨迹等更复杂模态，canonical space / instance space 的同步化是否还能保持稳定？

8.6 与训练时控制方法的分工

把本讲内容放回更完整的生成系统视角，可以得到一个很实用的判断：推理时引导并不是要取代 ControlNet、LoRA 或条件扩散模型，而是与它们分工协作。

路线	最擅长的问题	主要代价	更适合的场景
训练时控制	固定且高频的条件类型	需要数据、训练和维护多个模型	稳定生产环境、低延迟服务
推理时引导	临时、多变、组合式约束	采样更慢、调参更敏感	研究探索、长尾需求、复杂约束拼装
混合路线	训练时给粗结构，推理时做精修	系统设计更复杂	高质量生成与灵活控制同时要求较高的任务

表 3: 训练时控制与推理时引导的职责边界

这张表的核心启示是：如果某种条件几乎每次都出现，最好把它吸收到训练阶段；如果条件是临时的、跨任务的、难以收集监督数据的，就更适合保留到推理阶段解决。

8.7 拓展阅读

- **DPS 原始论文**：Chung et al., “Diffusion Posterior Sampling for General Noisy Inverse Problems,” ICLR 2023
- **Test-time scaling survey**：讲者推荐了一篇关于 test-time scaling 技术的综述论文，涵盖了扩散模型和流模型领域的多种技术
- **SyncDiffusion**：Lee et al., “SyncDiffusion: Coherent Montage via Synchronized Joint Diffusions,” NeurIPS 2023——全景图像同步化生成

- **Visual Anagrams**: Geng et al., “Visual Anagrams: Generating Multi-View Optical Illusions with Diffusion Models,” CVPR 2024——歧义图像生成
- **TEXTure**: Richardson et al., “TEXTure: Text-Guided Texturing of 3D Meshes,” SIGGRAPH 2023——利用 2D 扩散模型进行 3D 纹理生成
- **RLHF for Diffusion**: Fan et al., “DPOK: Reinforcement Learning for Fine-tuning Text-to-Image Diffusion Models”——将 RLHF 思想引入扩散模型训练